

ICARUS: Intelligence-driven Contextual Assessment of Risks in Unseen Scenarios

Ricardo Oliveira
University of Minho
pg54177@alunos.uminho.pt

João Marco C. Silva
University of Minho &
INESC TEC
joaomarco@di.uminho.pt

Abstract—Cyber Threat Intelligence (CTI) provides organizations with valuable information on indicators, attack patterns, and threat actors, but its operational use in detection pipelines is largely limited to atomic IOC matching and manual enrichment. Few deployed systems autonomously traverse structured CTI relationship graphs together with telemetry from monitored assets to generate contextual alerts for previously unseen threats without manually defined rules. This paper presents ICARUS, an open-source CTI-driven detection system that addresses this gap by integrating STIX feeds with real-time telemetry from monitored assets and traversing their relationship graphs to identify threats that are difficult to surface with IOC-based approaches. ICARUS assesses risk using a model that combines threat criticality, contextual depth, and temporal decay, and autonomously adapts data collection as the threat landscape evolves. Evaluated through nine functional tests across five independent deployments, preliminary results indicate that ICARUS enables automatic detection of previously unseen threats without manual intervention, pointing to CTI graph traversal as a promising autonomous driver of proactive cyber defense.

Index Terms—Cyber Threat Intelligence, STIX, Threat Detection, Risk Assessment, Security Monitoring

I. INTRODUCTION

The complexity and volume of cyber threats increasingly exceed the capacity of traditional security monitoring systems. Modern Security Information and Event Management (SIEM) platforms and Intrusion Detection Systems (IDSs) frequently rely on static rules and signature-based mechanisms, limiting their adaptability to novel, rapidly evolving threats. This architectural rigidity renders them fundamentally reactive, as detection logic is tied to known threat patterns, leaving systems structurally unable to identify threats for which no prior signature or rule has been defined.

Previous efforts standardized threat intelligence sharing through formats such as Structured Threat Information Expression (STIX) and Trusted Automated eXchange of Intelligence Information (TAXII) [1], [2], yet a gap persists. Although current platforms perform CTI enrichment and atomic Indicators of Compromise (IOC) matching, few autonomously traverse structured CTI relationship graphs, using telemetry captured from monitored assets to produce contextual alerts for previously unseen threats without manually defined rules.

This gap is largely structural rather than incidental. Existing detection architectures are designed around predefined signatures and rule sets and tend to lack the mechanisms

to dynamically bind multi-source threat context to locally collected observables in real time.

This gap is further compounded by an accelerating threat landscape. With nearly 50,000 new CVEs published in 2025 alone [3] and the average time-to-exploit continuing to shrink [4], telemetry collected before relevant intelligence is published is routinely discarded rather than retrospectively evaluated. As a result, organizations are dependent on reactive workflows that cannot proactively identify emerging threats.

This paper addresses this gap through ICARUS (*i.e.*, Intelligence-driven Contextual Assessment of Risks in Unseen Scenarios), a CTI-based alerting system integrating modern CTI standards into dynamic detection pipelines. ICARUS makes four key contributions.

- A modular architecture integrating real-time telemetry and structured CTI to enable autonomous, contextual alerts;
- A complete proof of concept ingesting STIX feeds and correlating external observations with the local threat landscape;
- Demonstration of how dynamic CTI correlations enable early detection of emerging threats without manual rule definitions;
- Autonomous, CTI-driven adaptation of data collection strategies without manual intervention.

The prototype implementation, together with all configuration files and testbed artifacts, is released as an open-source project¹, with the aim of fostering further research in autonomous, intelligence-driven cyber defense.

The remainder of this paper is organized as follows. Section II reviews related work on CTI standards and sharing ecosystems and identifies the structural limitations that motivate this work. Section III presents the ICARUS system architecture, including its design goals and detection pipeline. Section IV describes the proof of concept implementation. Section V evaluates the system through functional testing. Section VI concludes the paper and outlines directions for future work.

II. RELATED WORK AND LIMITATIONS

The STIX language [1], [5] provides a comprehensive schema for representing IOCs, attack patterns, and threat actor

¹<https://github.com/RicAlvesO/ICARUS>

behaviors, distinguishing itself from narrower formats such as CyBOX [6] and CVE [7] by modeling a broad spectrum of threat intelligence while remaining both human- and machine-interpretable. The TAXII protocol [2] specifies the primary transport mechanism for STIX data, while platforms such as MISP [8] and AlienVault OTX [9] support community-based sharing [10]. Research has further advanced CTI production and enrichment through privacy-preserving generation [11], automated pipelines [12], and graph-based enhancement [13], [14], while quality assessments [15], [16] and heterogeneity analyses [17], [18] document persistent obstacles in operational CTI adoption.

Despite these advances, the integration of CTI into automated detection logic beyond atomic IOC matching remains largely conceptual or requires substantial human oversight [19], [20]. In prevailing practice, SIEM and IDS platforms treat CTI as auxiliary enrichment, and the autonomous traversal of STIX relationship graphs to produce rule-free contextual alerts for previously unseen threats remains a largely unaddressed challenge. Timeliness is a compounding concern, as the decreasing average time-to-exploit [4] means delayed assimilation of new indicators leaves organizations exposed, while heterogeneous data quality and source inconsistencies hinder reliable automation at scale. The following structural gaps summarize these shortcomings.

- *Static IOC matching* dominates current systems, preventing contextual or behavior-aware detection;
- *Lack of graph-based autonomous correlation*: existing systems cannot traverse STIX relationship graphs to connect telemetry from monitored assets with indirect threat associations, limiting detection to directly observable indicators;
- *Slow adaptation to newly published indicators* leaves defenders exposed during the window between threat publication and rule deployment;
- *High variability in CTI quality* and format consistency complicates automated use at scale;
- *Absence of adaptive evidence collection*: when a potential threat is identified, systems lack mechanisms to automatically deepen monitoring on affected assets to gather corroborating evidence.

III. SYSTEM ARCHITECTURE

This section describes the design goals and architectural components of ICARUS, detailing how each element contributes to the intelligence-driven detection pipeline.

A. Design Goals

Five objectives, derived from the limitations in Section II, guide the ICARUS design. The system must *ingest and normalize* both external CTI and local endpoint data into a unified intelligence graph, and *correlate beyond IOC matching* by leveraging STIX relationships and threat-actor context to detect threats even when explicit indicators are not directly observable. Correlated events are evaluated through a *risk-oriented scoring mechanism* that accounts for temporal decay,

threat criticality, and contextual depth. Each detection must produce *context-rich, actionable alerts* that expose the full reasoning vector to analysts. Finally, *autonomous adaptability* requires the system to continuously incorporate new intelligence and adjust data-collection strategies without manual intervention.

B. Architecture Overview

ICARUS follows a modular design centered on three core components.

- *CTI Providers*: External feeds supplying structured STIX data through TAXII-like interfaces, continuously updating the intelligence database as new threats emerge;
- *Endpoint Agents*: Lightweight processes deployed on monitored hosts, collecting local observables (*e.g.*, running processes, network connections, file hashes) and transmitting them to the main server. Agents support remote management for dynamic query updates;
- *Main Server*: The orchestration core, handling ingestion, correlation, risk assessment, and alert generation, and exposing an analyst-facing user interface.

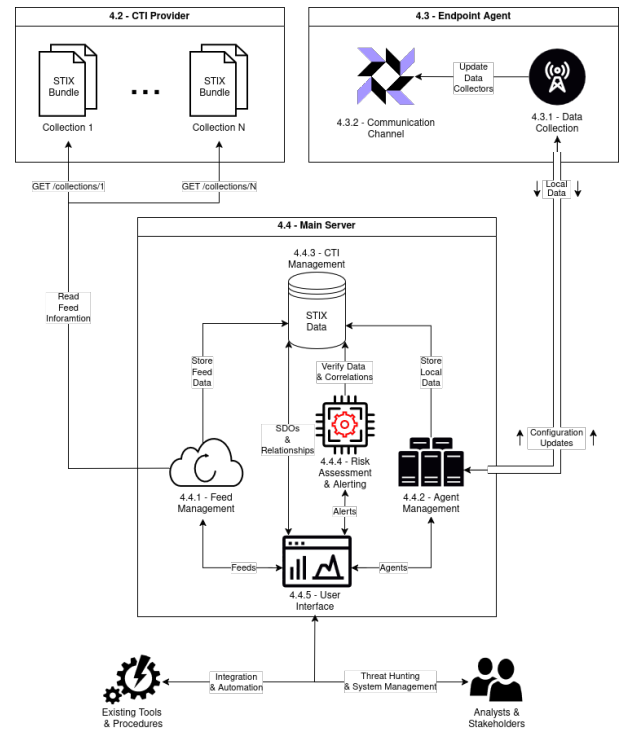


Figure 1. Overview of ICARUS architecture and component interactions.

Figure 1 illustrates these components and their interactions, forming the foundation of the intelligence-driven detection pipeline.

C. Detection Pipeline

The detection pipeline (represented in Figure 2) progresses through seven stages. (1) *CTI Feeds and Local Agents* supply the two complementary intelligence streams, with external providers delivering structured STIX bundles and endpoint



Figure 2. The ICARUS detection pipeline from data ingestion to incident response.

agents transmitting real-time host observables. (2) *CTI Ingestion* parses and normalizes both streams into a unified STIX-based representation with indexes supporting efficient querying. (3) *Verify Correlations to Agents* evaluates whether newly acquired intelligence links to monitored endpoints, leveraging STIX relationships to infer potential attack vectors. (4) *Identify Possible Threats* filters correlations by base threat risk and historical detections to reduce alert fatigue. (5) *Assess Risk* computes a dynamic risk score incorporating threat criticality, temporal decay, and contextual depth. (6) *Generate Alert* creates a structured alert when the risk score exceeds the defined threshold. (7) *Incident Response* presents prioritized, contextually enriched alerts to analysts through the user interface.

IV. PROOF OF CONCEPT

Based on the architecture described in Section III, a complete proof of concept was implemented. This section describes the key modules and their integration into the detection pipeline.

A. External Intelligence Ingestion

The first component is responsible for ingesting external threat intelligence in a structured and repeatable manner. Its design mirrors common practices in contemporary CTI sharing ecosystems, where the external provider is implemented to replicate the behavior of operational CTI services in a controlled environment, enabling consistent testing and validation of the ICARUS detection pipeline.

The CTI provider follows the principles of TAXII collections, offering a clear request-response model for retrieving intelligence bundles. Each collection is represented as a sequence of STIX bundles. A bundle groups multiple STIX objects alongside their relationships, enabling a cohesive representation of threat entities and their contextual links. Each bundle carries four core fields, namely *type* (identifies the object as a STIX bundle), *id* (a unique identifier), *objects* (an array of STIX objects such as indicators, malware, or network artifacts), and *relationships* (objects describing how the entities connect and interact). This structure combines atomic indicators with contextual relationships, allowing ICARUS to infer links between locally observed objects and possible threats rather than relying solely on isolated indicator matching.

This component is configured with a list of collections stored as JSON files. To emulate dynamic intelligence streams, bundles can be exposed incrementally or updated over time, simulating continuous evolution in the threat landscape.

B. Local Data Collection

The endpoint agent periodically collects host telemetry using *osquery*'s SQL-based interface, which maps directly

to STIX Domain Objects (SDOs). Agents support remote reconfiguration from the server, enabling the data collection strategy to evolve dynamically in response to emerging threats.

```

SELECT f.path, f.size,
       f.atime, f.ctime, f.mtime,
       h.md5, h.shal, h.sha256
FROM file f JOIN hash h
ON f.path = h.path
WHERE (f.mode & 73) != 0
AND f.directory IN ('/tmp');
  
```

Listing 1. Example *osquery* query retrieving executables in /tmp with cryptographic hashes.

The JOIN in Listing 1 aggregates structural and integrity data into a single enriched record, unambiguously identifying each file across systems and CTI feeds.

C. Server-Agent Communication

The server-agent channel serves two complementary functions. The first, *data transmission*, delivers collected host telemetry to the server in a structured and timely manner. The second, *remote management*, enables the server to push updated collector queries and configuration changes to agents at any time. This bidirectional design ensures continuous delivery of observables while preserving the flexibility to adapt data collection in response to the evolving threat landscape.

The channel is implemented as a custom protocol layered over TLS, providing confidentiality and integrity of transmitted data. Messages are encoded as JSON objects, a format that is human-readable, broadly supported, and directly convertible to SDO structures on arrival. The protocol defines four message types.

- *data*: agent sends collected query results, encapsulated for direct conversion to SDOs;
- *upd*: server pushes new or modified collector queries for dynamic reconfiguration;
- *err*: either side reports anomalies such as malformed messages or collection failures;
- *ack*: confirms that a message has been successfully received and processed.

On the agent side, two dedicated loops manage communication. The *sender loop* periodically executes all active queries and transmits results as *data* messages at the configured interval. The *listener loop* is reactive: upon receiving an *upd* message, it updates the active query set without interrupting the sender, enabling seamless remote reconfiguration.

D. Data Storage and Correlation

The main server's data storage serves as the unified repository for all threat intelligence, whether from external feeds or local agents. All data is stored as structured STIX objects and relationships, ensuring consistency across heterogeneous sources.

Before insertion, incoming data undergoes parsing and normalization to enforce a uniform structure and deduplicate overlapping reports. ICARUS applies a canonical fingerprinting mechanism where non-essential metadata is stripped from each object, and the remaining semantic content is serialized into a sorted JSON representation, then hashed with SHA-256

to produce a unique fingerprint of the object's meaning rather than its original form. This allows duplicates from different providers to be merged by fingerprint lookup alone, without exhaustively comparing every field, maintaining a coherent and non-redundant intelligence database.

Each stored object is complemented with essential metadata, including its identifier, type, Traffic Light Protocol (TLP) sensitivity marking, initially assigned risk score, source origin, and modification history. These fields support traceability and filtering, enabling the system to maintain contextual awareness as objects evolve over time.

Beyond individual SDOs, the storage tracks all relationships between objects, including those linking local agents to external intelligence. With relationships from external CTI stored alongside those derived from local endpoint data, the system constructs a comprehensive graph of interconnected entities. This intelligence network captures not only direct indicator matches that may signal compromise, but also indirect associations spanning multiple hops, such as a host reporting a network connection that ultimately traces back to known malicious infrastructure through a chain of related STIX objects. These relational paths form the input to the risk assessment stage.

E. Risk Assessment

ICARUS traverses the intelligence graph from each agent node to identify all *threat vectors*, defined as paths leading to SDOs with non-zero risk scores. Vectors that pass a base risk threshold and are not redundant with historical detections are then scored. The risk formula is derived as follows. The path p alternates between nodes and relationships, starting with the agent node, so its total length is $L = 1 + 2d$ and depth $d = (L - 1)/2$. Given base threat risk R and configurable depth multiplier f , the initial adjusted value is $R \cdot f$, capped to the 0–100 scale,

$$\text{vector_risk} = \min(R \cdot f, 100)$$

Dividing by depth d ensures that nodes further from the agent contribute proportionally less, while leaving the score unchanged when $d = f$ (i.e., when the path length matches the multiplier),

$$\text{vector_risk} = \min\left(\frac{R \cdot f}{d}, 100\right)$$

Substituting $d = (L - 1)/2$ yields the operational formula used in the proof of concept,

$$\text{vector_risk} = \min\left(\frac{R \cdot f \cdot 2}{L - 1}, 100\right) \quad (1)$$

Scores are further modulated by source trustworthiness and temporal decay, preventing stale indicators from dominating. The combination of these factors creates a nuanced risk assessment that is deterministic yet adaptive to the dynamic nature of cyber threats.

F. Alerting and Adaptive Collection

When a vector's risk exceeds the configured threshold, ICARUS generates a structured alert containing the affected

host, trigger object, risk score, detection timestamp, and the full correlation chain with all involved SDOs. The alerting module additionally monitors mean risk per SDO type, activating or deactivating the corresponding endpoint collectors autonomously when a threshold is crossed, creating a closed feedback loop that adapts data collection depth to the evolving threat landscape without manual intervention.

G. Implementation Summary

ICARUS is implemented in Python using a graph-oriented storage layer that natively represents STIX relationships, enabling efficient path traversal during correlation and risk assessment. The main server exposes a REST API and a web-based analyst interface. All components are fully containerized for consistent deployment and replication.

V. SYSTEM EVALUATION

This section evaluates ICARUS through functional testing, assessing whether each pipeline component operates as intended and whether the system as a whole fulfills its stated design objectives.

A. Test Environment

The proof of concept was deployed in a reproducible environment provisioned with Ansible, Dockerfiles, and Docker Compose. Figure 3 shows the setup, where the internal network hosts two monitored endpoints (10.10.0.3, 10.10.0.4) and the ICARUS server (10.10.0.2), while the external network contains a trusted CTI provider (10.20.0.2) and a malicious host (10.20.0.3) for correlation testing. Each entity runs in an isolated Docker container, ensuring repeatable interactions.

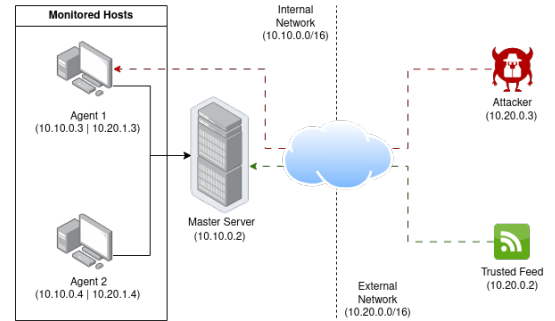


Figure 3. Experimental test environment with internal and external network segments.

B. Testing Methodology

The evaluation of ICARUS focuses on functional correctness and behavioral validation rather than benchmarking raw throughput or comparing detection rates against established SIEM or IDS baselines, tasks that would require a fundamentally different experimental setup and a shared ground truth. The goal is to establish that each component of the pipeline operates as intended, and that the system as a whole fulfills its stated objectives across three evaluation areas: (i) autonomous

CTI ingestion and correlation; (ii) autonomous contextualized alerting; and (iii) adaptive detection.

Nine functional tests are organized into three corresponding test groups, each executed five times across independently redeployed environments to confirm consistency. Table I summarizes the outcomes.

The evaluation parameters were configured as follows. The alerting risk threshold was set to 40, following Kent’s scale of estimative probabilities [21], below which a detected correlation is considered insufficiently certain to warrant analyst attention. The depth multiplier was set to $f = 3$, meaning correlations up to three hops from the agent carry the full weight of the threat’s base score, while deeper paths are attenuated proportionally. The decay rate was configured to reduce each object’s risk score by one point every 30 seconds, with history entries created at each multiple-of-ten reduction.

Table I
SUMMARY OF FUNCTIONAL TEST OUTCOMES

Test	Objective	Result
T_1	External CTI Ingestion	Pass
T_2	Local Data Collection	Pass
T_3	Basic Correlation	Pass
T_4	Risk Assessment Accuracy	Pass
T_5	Alert Prioritization	Pass
T_6	Temporal Decay	Pass
T_7	Alert Contextualization	Pass
T_8	Continuous Evolution	Pass
T_9	Autonomous Collector Management	Pass

C. Test Group 1: Data Collection and Correlation

This test group validates that ICARUS can autonomously acquire and unify intelligence from both external CTI providers and data captured from monitored assets, then identify meaningful correlations between them. The following examples illustrate how each objective was met. Autonomous CTI ingestion (T_1) is exemplified by the server polling and assimilating a STIX bundle without any manual step. Autonomous local data collection (T_2) is illustrated by agents self-configuring upon connection and immediately reporting endpoint observables. Rule-free correlation (T_3) is demonstrated by a scenario in which Agent 1 established a connection to a host reported in the CTI feed. Without any manually defined rule, the server identified the relationship and produced the correlation graph shown in Figure 4.



Figure 4. Correlation graph linking Agent 1 to malicious infrastructure via observed network traffic, generated without any rule definition.

D. Test Group 2: Alerting and Risk Assessment

This test group validates that identified correlations are correctly translated into prioritized, accurately scored, and

contextually rich alerts. Selected scenarios from each test are presented here as representative examples of the behaviors confirmed. Risk scoring accuracy (T_4) is illustrated by six hand-computed scenarios compared against system output. All six matched exactly, including two edge cases that correctly produced no alert, as shown in Table II. Alert prioritization (T_5), temporal decay (T_6), and contextualization (T_7) are exemplified by, respectively: alerts appearing sorted by descending risk score, scores being recomputed live at alert time rather than cached, and each alert exposing the full correlation chain with all involved SDOs, a capability that static rule-based systems typically do not provide.

Table II
EXPECTED VS. ACTUAL RISK SCORES ($f = 3$, THRESHOLD = 40)

Case	R_0	Δ_T (min)	d	Exp.	Act.
1	80	1	1	100	100
2	50	3	2	66	66
3	100	0	6	50	50
4	75	10	3	55	55
5	35	0	3	–	–
6	100	5	9	–	–

E. Test Group 3: Autonomy and Adaptability

This test group validates the system’s ability to operate adaptively over time. The scenarios below are representative examples that illustrate the adaptive behaviors observed across all test runs. Retrospective detection (T_8) is exemplified by a case in which a connection established before any matching CTI existed was automatically flagged once a subsequent feed update introduced the relevant threat actor, without any analyst intervention; a second agent connecting to the same host after the update triggered a real-time alert, illustrating that both modes coexist in the same pipeline.

Autonomous collector management (T_9) is illustrated by a scenario in which rising mean risk scores caused the server to autonomously activate the appropriate endpoint collectors and distribute updated configurations to all agents (Figure 5). When temporal decay subsequently lowered the mean risk below the thresholds, collectors were deactivated automatically. This bidirectional feedback loop, where intelligence drives collection and collection enriches the intelligence graph is the centerpiece of the system’s adaptive capability.

F. Discussion

All nine tests passed across all five independent deployments, indicating that ICARUS meets the objectives of all three test groups. The data collection and correlation tests showed that the system can autonomously ingest structured CTI, self-configure endpoint agents, and identify threat relationships without any manually defined rule, extending static IOC matching with graph-based STIX traversal. The alerting and risk assessment tests showed that the scoring model, combining base risk, temporal decay, and contextual depth, produces accurate and deterministic results across the scenarios considered, including edge cases that correctly yield no alert. The autonomy and adaptability tests indicated that pre-ingestion telemetry is retrospectively evaluated against newly


```

2025-10-27 22:44:40,856 - INFO - Successfully read feed 'trusted1'.
2025-10-27 22:44:40,857 - INFO - Added new object with ID process--
1e42945a-6715-4802-aa85-6b0039995b7c from feed 'trusted1'.
2025-10-27 22:45:01,657 - INFO - Received message from ('10.10.0.3'
, 44308)
2025-10-27 22:45:01,657 - INFO - Processing input from ('10.10.0.3'
, 44308): list_network_connections
2025-10-27 22:45:01,683 - INFO - Received message from ('10.10.0.4'
, 45899)
2025-10-27 22:45:01,683 - INFO - Processing input from ('10.10.0.4'
, 45899): list_network_connections
2025-10-27 22:45:10,887 - INFO - Processing alerts for agent identi
ty--11ab5fbb-232a-4242-b516-5e4c59ad96c2
2025-10-27 22:45:10,888 - INFO - Processing alerts for agent identi
ty--e38189cd-70d8-4755-a7c5-139cfd7ef8e4
2025-10-27 22:45:10,888 - INFO - Current mean risks by type: {"thre
at-actor": 96.0, "ipv4-addr": 36.0, "domain-name": 26.0, "relations
hip": 26.0, "process": 49.0}
2025-10-27 22:45:10,888 - INFO - Query 'list_running_processes' ena
bled state changed from False to True

```

Figure 5. Server log showing autonomous collector activation as mean SDO risk thresholds are crossed.

published intelligence, closing the detection window, and that data collection strategies adapt autonomously as the threat landscape evolves without any manual intervention.

The evaluation scope is bounded by the controlled environment, where throughput and correlation latency at enterprise scale, and the representativeness of real-world CTI feed variability, remain uncharacterized. These are deliberate trade-offs of a feasibility-oriented evaluation, as confirming functional correctness at small scale is the necessary precondition for the optimization and hardening work a production deployment would require.

VI. CONCLUSIONS

This paper presented ICARUS, a modular, open-source CTI-driven detection system that operationalizes Cyber Threat Intelligence as an active and autonomous driver of contextual threat detection. The proposed architecture integrates real-time endpoint telemetry with structured external intelligence through a graph-based STIX correlation engine, identifying threats that are difficult to surface with static IOC-matching approaches, including indirect associations spanning multiple relationship hops.

Beyond the architecture, the implementation introduced several key mechanisms, namely a canonical fingerprinting strategy that deduplicates overlapping intelligence from heterogeneous sources, a bidirectional server-agent protocol that enables remote reconfiguration of endpoint collectors without interrupting ongoing telemetry collection, a risk scoring model that accounts for threat criticality, contextual depth, and temporal decay, and an autonomous feedback loop that activates and deactivates collectors in proportion to the evolving threat landscape. All nine functional tests were satisfied across five independently redeployed environments, offering initial evidence for both the design objectives and the feasibility of CTI as a practical, autonomous detection pipeline.

Future work will address scale characterization under enterprise-grade telemetry volumes, risk model refinement through machine learning calibration, and operational integration with SOAR platforms and analyst-verdict feedback mechanisms. ICARUS is released as an open-source project,

enabling the community to build upon, validate, and extend the presented work.

Acknowledgment: This work is co-funded by the European Regional Development Fund (ERDF) through the NORTE 2030 Regional Programme under Portugal 2030, within the scope of the project MOBOT2, reference 24264 (NORTE2030-FEDER-02961400).

REFERENCES

- [1] S. Barnum, "Standardizing cyber threat intelligence information with the structured threat information expression (stix)," *Mitre Corporation*, vol. 11, pp. 1–22, 2012.
- [2] J. Connolly, M. Davidson, and C. Schmidt, "The trusted automated exchange of indicator information (taxii)," *The MITRE Corporation*, pp. 1–20, 2014.
- [3] N. I. of Standards and Technology, "National vulnerability database (NVD) dashboard," <https://nvd.nist.gov/general/nvd-dashboard>, 2026, accessed: 2026-05-29.
- [4] cybermindr, "The race against exploitation: Average time-to-exploit in 2025," 2025, accessed: 2025-10-25. [Online]. Available: <https://www.cybermindr.com/blog/average-time-to-exploit-in-2025/>
- [5] B. Jordan, R. Piazza, and T. Darley, "Stix version 2.1," OASIS Standard, June 2021, standards Track Work Product. Copyright © OASIS Open 2022. All Rights Reserved. [Online]. Available: <https://docs.oasis-open.org/cti/stix/v2.1/os/stix-v2.1-os.html>
- [6] S. Barnum, R. Martin, B. Worrell, and I. Kirillov, "The cybox language specification," *The MITRE Corporation*, 2012.
- [7] R. Martin, S. Christey, D. Baker, and M. Corporation, "The common vulnerabilities and exposures (cve) initiative," *MITRE Corporation*, 2002.
- [8] The MISP Project, "Misp: Malware information sharing platform," 2025, accessed: 2025-01-24. [Online]. Available: <https://www.misp-project.org/>
- [9] I. LevelBlue, "Alienvault otx," 2025, accessed: 2025-01-24. [Online]. Available: <https://otx.alienvault.com/>
- [10] P. Kampanakis, "Security automation and threat information-sharing options," *IEEE Security & Privacy*, vol. 12, no. 5, pp. 42–51, 2014.
- [11] F. Sadique, S. Cheung, I. Vakili, S. Badsha, and S. Sengupta, "Automated structured threat information expression (stix) document generation with privacy preservation," in *2018 9th IEEE Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, Nov. 2018, p. 847–853.
- [12] Z. Iqbal, Z. Anwar, and R. Mumtaz, "Stixgen-a novel framework for automatic generation of structured cyber threat information," in *2018 International Conference on Frontiers of Information Technology (FIT)*. IEEE, 2018, pp. 241–246.
- [13] F. Marchiori, M. Conti, and N. V. Verde, "Stixnet: A novel and modular solution for extracting all stix objects in cti reports," in *Proceedings of the 18th International Conference on Availability, Reliability and Security*, 2023, pp. 1–11.
- [14] S.-S. Chen, R.-H. Hwang, A. Ali, Y.-D. Lin, Y.-C. Wei, and T.-W. Pai, "Improving quality of indicators of compromise using stix graphs," *Computers & Security*, vol. 144, p. 103972, 2024.
- [15] D. Schlette, F. Böhm, M. Caselli, and G. Pernul, "Measuring and visualizing cyber threat intelligence quality," *International Journal of Information Security*, vol. 20, no. 1, p. 21–38, Feb. 2021.
- [16] D. Schlette, M. Vielberth, and G. Pernul, "Cti-soc2m2—the quest for mature, intelligence-driven security operations and incident response capabilities," *Computers & Security*, vol. 111, p. 102482, 2021.
- [17] A. Ramsdale, S. Shialeles, and N. Kolokotronis, "A comparative analysis of cyber-threat intelligence sources, formats and languages," *Electronics*, vol. 9, no. 5, p. 824, 2020.
- [18] D. Chismon and M. Ruks, "Threat intelligence: Collecting, analysing, evaluating," *MWR InfoSecurity Ltd*, vol. 3, no. 2, pp. 36–42, 2015.
- [19] Y. He, E. Inglut, and C. Luo, "Malware incident response (ir) informed by cyber threat intelligence (cti)," *Science China. Information Sciences*, vol. 65, no. 7, p. 179105, 2022.
- [20] Y. Guo, "A review of machine learning-based zero-day attack detection: Challenges and future directions," *Computer communications*, vol. 198, pp. 175–185, 2023.
- [21] S. Kent, "Words of estimative probability," 1964.